Random Forest Self-Training: Do Confidence Thresholds Affect Accuracy

Name: Anusha Baddam **Id: 24127511**

GitHub: Random_Forest_Self-Training-_Do_Confidence_Thresholds_Affect_Accuracy.git

Introduction:

Random Forest is a supervised machine learning algorithm that builds many decision trees and combines their outputs to make a final prediction. Each tree sees a random subset of the training data and, at each split, a random subset of the features. For classification, every tree votes for a class and the forest predicts the class with the most votes. For regression, it averages the predictions from all trees. Because the trees are different from each other, their individual mistakes tend to be cancled out. This makes the overall model more stable and less likely to overfit than a single deep decision tree. Random Forests can handle non-linear decision boundaries and usually work well with only a few main hyperparameters, such as the number of trees and the maximum depth.

In many real problems it is expensive to label data, so we often have a small, labeled set and a much larger unlabeled set. Semi-supervised learning tries to use both labeled and unlabeled data together. Self-training is one of the simplest semi-supervised methods. In self-training with Random Forest, we first train the model only on the small, labeled subset. We then use this model to predict class probabilities for unlabeled examples. Points where the model is very confident (above a chosen probability threshold) are given pseudo-labels and added to the labeled set. The Random Forest is retrained on this expanded dataset, and the process can be repeated for several rounds.

The confidence threshold plays an important role in this process. A low threshold adds many pseudo-labels, but some of them may be wrong, which can hurt test accuracy. A high threshold adds fewer but more reliable pseudo-labels, so the model is safer but may learn less from the unlabeled data. This tutorial explores how changing that threshold affects the performance of a self-training Random Forest.

In semi-supervised learning, self-training and pseudo-labeling are widely used because they are simple and model-agnostic: the model uses its own high-confidence predictions on unlabeled data as additional training labels. This idea appears in early self-training work and in later pseudo-labeling methods used with deep networks, where confidence thresholds or top-k selection are critical to avoid reinforcing wrong labels. Our tutorial follows this line of work in a

simple Random Forest setting and focuses on how the confidence threshold controls which unlabeled points are allowed to influence the model.

Methodology:

In this tutorial, we show how to use self-training with a Random Forest on a random two-class problem. We start by generating the make moons dataset from scikit-learn with 1,000 samples and some noise, which creates two interlocking half-moons that are not linearly separable. Each point has two features x1, x2 and belongs to one of two classes shown as blue and red clusters in the scatter plot, so the decision boundary must curve to follow the moon shapes rather than being a straight line. We then split the data into 80% training and 20% test sets, keeping the class balance the same. Because the make moons data has a strongly curved decision boundary, simple linear models struggle, whereas tree-based ensembles can approximate the boundary by combining many axis-aligned splits. Random Forest is therefore a natural choice to study how self-training behaves on a non-linear problem.

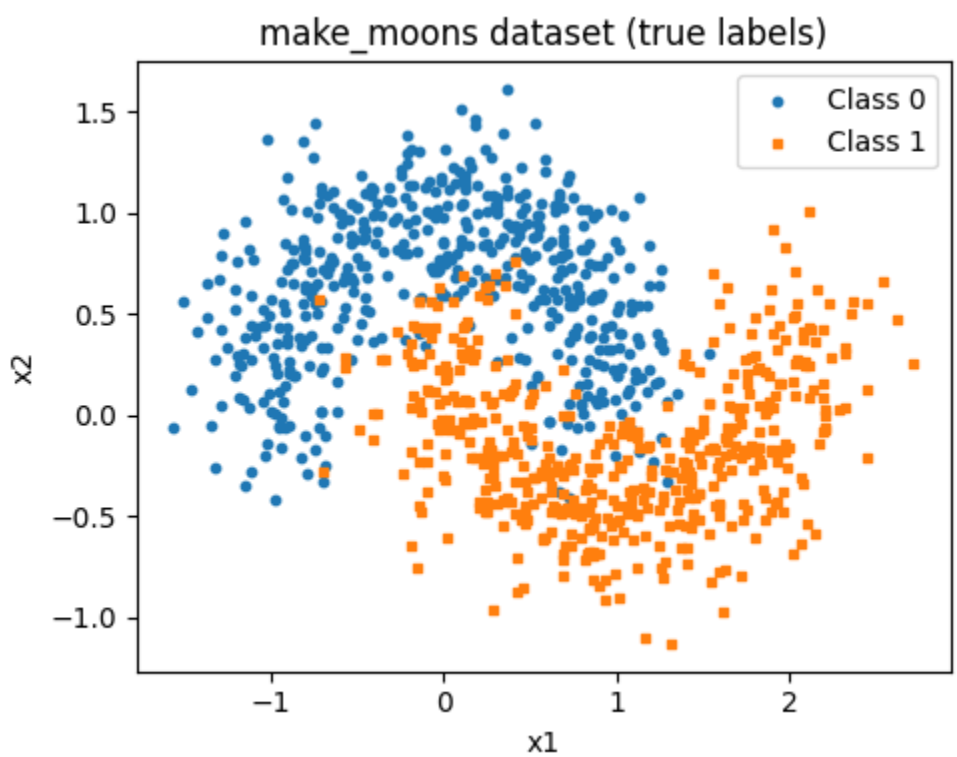

Figure 1: Scatter plot of the make moons dataset showing two interlocking half-moons (classes 0 and 1) with different colours and marker shapes.

To mimic a semi-supervised setting, we randomly pick 100 training points and keep their labels, and we treat the remaining 700 training points as unlabeled. First, we train a normal Random Forest classifier with 200 trees only on the 100 labeled points and measure its test accuracy. This acts as our supervised baseline.

Next, we turn this into a self-training loop. In each round, we fit a Random Forest on the current labeled set, use it to predict class probabilities for all unlabeled points, and then select only those points where the model is very confident (for example, maximum class probability above 0.8, 0.9, or 0.95). We give these points pseudo-labels from the model, add them to the labeled set, and remove them from the unlabeled pool. We repeat this process until no more points pass the threshold or we reach 10 iterations and we then compare the final test accuracy for each threshold with the original baseline. Unlike a standard supervised Random Forest tutorial, this notebook implements a full semi-supervised self-training loop, including management of labeled vs unlabeled pools, iterative re-training, and explicit control of confidence thresholds (0.8, 0.9, 0.95). This makes the exercise technically more demanding than training a single classifier once.

Results:

Note: *Because we randomly select which 100 training points are treated as labeled, the exact train/test split into labeled and unlabeled sets can change between runs. As a result, the reported accuracies and the number of pseudo-labels added at each iteration may vary slightly when the notebook is re-run, even though the overall patterns and conclusions remain the same.*

On this dataset, the Random Forest trained only on the 100 labeled points already achieves high test accuracy (around 0.97), so it is a strong supervised baseline. When we apply self-training with a relatively low threshold such as 0.8, the model quickly adds many pseudo-labeled points from the unlabeled pool, but some of these labels are incorrect, so the test accuracy can fluctuate or slightly decrease compared to the baseline. With a threshold of 0.9, fewer points are added in each iteration, but the pseudo-labels are more reliable and the final accuracy remains close to the baseline while using more data. For a very high threshold like 0.95, the model becomes conservative, adding only a small number of highly confident points; accuracy is stable but there is little gain from the unlabeled data because the labeled set grows slowly. These experiments illustrate that the confidence threshold controls a trade-off between using more unlabeled data and avoiding noisy pseudo-labels.

Threshold	Test accuracy	Final labeled points	Remaining unlabeled points	Short comment
Baseline	0.940	100	700	Strong supervised model, uses only a small labeled set.
0.8	0.955	774	26	Highest use of unlabeled data; accuracy slightly improves but may be less stable due to noisy pseudo-labels.

0.9	0.935	694	106	Balanced: adds many points while keeping labels relatively clean; accuracy close to baseline.
0.95	0.940	562	238	Very conservative; almost only adds very confident points, accuracy stable but limited benefit from unlabeled data.

This table can be used in teaching to quickly show the trade-off between "how much unlabeled data we use" and "how safe the accuracy is" for different thresholds.

When we compare the test predictions of the baseline Random Forest and the self-training Random Forest with a threshold of 0.9. On both plots, red circles and blue circles show correctly classified points from the two classes, while blue crosses mark the few misclassified points. The baseline model achieves accuracy 0.970, slightly higher than the self-training model's 0.945, which suggests that on this simple dataset self-training with this threshold does not improve performance over the already strong supervised model.

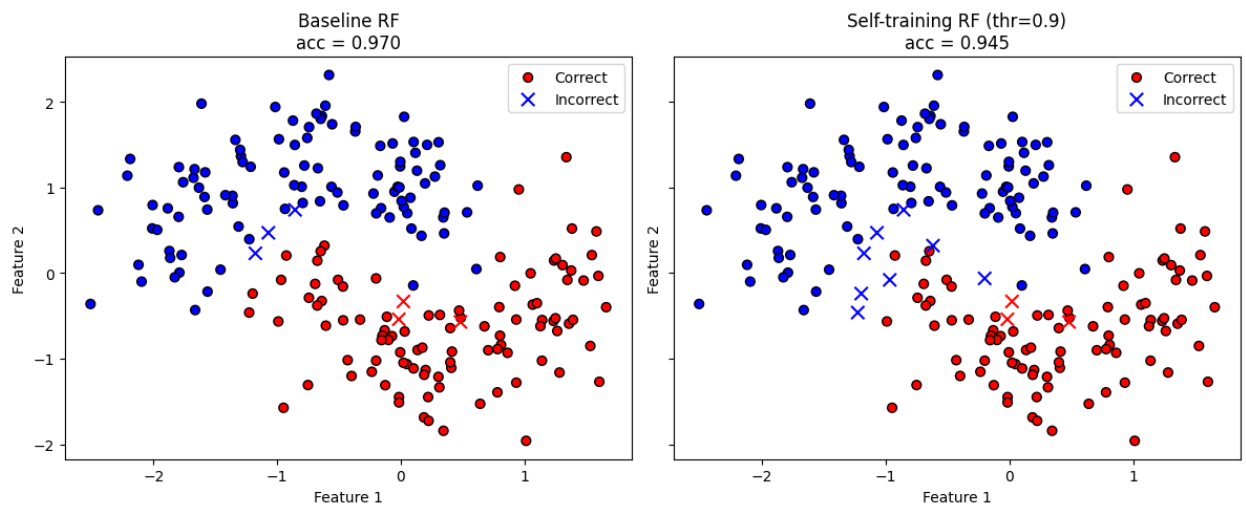

Figure 2: Test predictions of the baseline Random Forest and the self-training model (threshold 0.9), where correctly classified points use circles and misclassified points use crosses.

How the Threshold Effects Accuracy and the Model?

The confidence threshold mainly affects which pseudo-labels are added, so it changes both accuracy and model behaviour's.

- With a low threshold (for example 0.8), many unlabeled points are added at each iteration, including points the model is not very sure about. This quickly increases the size of the labeled set, but it also introduces more wrong pseudo-labels, which can slightly reduce or

destabilise test accuracy as these errors get reinforced. The model's decision boundary may become more irregular because it tries to fit noisy labels.

- With a medium threshold (around 0.9), fewer points are added per iteration, but they are more likely to be correct. Accuracy stays close to the supervised baseline while still benefiting from extra data, and the decision boundary is refined in a more stable way.
- With a very high threshold (for example 0.95), the model only adds very confident points, so almost all pseudo-labels are correct but the labeled set grows slowly. Accuracy is safe (rarely worse than the baseline), yet there is limited improvement because most unlabeled data are never used.

Takeaways:

These experiments show that there is no single "best" threshold; instead, it depends on whether we care more about using as much unlabeled data as possible or about keeping labels very clean. On our simple make moons example, the supervised Random Forest baseline is already strong, so self-training with aggressive (low) thresholds can slightly harm accuracy by introducing noisy pseudo-labels, while moderate or high thresholds keep performance close to the baseline but offer only small gains. In harder real-world problems where the initial labeled set is weaker, carefully tuning the threshold is likely to matter more, because it controls how quickly the model can "teach itself" without drifting due to wrong labels.

Conclusion:

In this tutorial, we used a Random Forest with self-training to explore how the confidence threshold controls the use of unlabeled data on a simple make moons task. The baseline Random Forest trained on only 100 labeled points already reached high accuracy, and self-training with a threshold of 0.9 produced a slightly lower accuracy while adding extra pseudo-labeled points. This shows that when the supervised model is already strong and the dataset is relatively easy, aggressive use of unlabeled data does not necessarily help and can even hurt if noisy pseudo-labels are added. In more challenging problems with fewer labels or more complex structures, carefully tuning the threshold is likely to be more beneficial, as it balances learning from additional data against the risk of propagating errors.

Learning objectives:

- Understand Random Forest classification and why it works on non-linear data.
- Implement and experiment with self-training using confidence thresholds.
- Interpret how pseudo-labels from unlabeled data influence accuracy.

References:

- Sayed Ebad. "Mastering Random Forest: A Comprehensive Guide with Detailed Math and Code." *Medium*, 2023. Available at:

https://medium.com/@sayedebad.777/mastering-random-forest-a-comprehensive-guide-with-detailed-math-and-code-60aebffb2b7f.

- Cheng, S.-H. "Unboxing the Random Forest Classifier: The Threshold Distributions." *The Airbnb Tech Blog*, 2015. Available at: https://medium.com/airbnb-engineering/unboxing-the-random-forest-classifier-the-threshold-distributions-22ea2bb58ea6.